

SHA-1 (Secure Hash Algorithm 1)

Aim

To implement SHA-1, a cryptographic hash that maps any message to a fixed 160-bit (40 hex character) digest. The message is padded to a multiple of 512 bits, split into blocks, and each block is run through 80 rounds that mix five 32-bit chaining registers. It is one-way (the input cannot be recovered from the digest) and exhibits the avalanche effect (a one-bit change flips about half the output bits). SHA-1 is now broken for security use, but remains a clean study of how a hash is built.

CRYPTOGRAPHIC HASH · ONE-WAY COMPRESSION

Type a novel or a single space. It leaves as 160 bits.

SHA-1 crushes any input — one byte or a gigabyte — into a fixed 40-hex fingerprint, and there's no way back. The message is padded to a multiple of 512 bits, cut into blocks, and stirred through 80 rounds that mix five 32-bit registers. Change one letter below and watch roughly half the output flip.

DIGEST · 160-BIT FINGERPRINT43 BYTES IN

2	f	d	4	e	1	c	6	7	a	2	d	2	8	f	c	e	d	8	4
9	e	e	1	b	b	7	6	e	7	3	9	1	b	9	3	e	b	1	2

Avalanche edit one character and roughly half of the 160 output bits should move.

01 MESSAGEHASHED LIVE AS YOU TYPE

plaintext (UTF-8)

The quick brown fox jumps over the lazy dog

Flip last character

Clear

Copy digest

Known check — "abc" hashes to a9993e36_9cd0d89d.

Inside the machine

1 · PADDING → MULTIPLE OF 512 BITS

54 68 65 20 71 75 69 63 6b 20 62 72 6f 77 6e 20 66 6f 7
8 20 6a 75 6d 70 73 20 6f 76 65 72 20 74 68 65 20 6c 61
7a 79 20 64 6f 67 80 00 00 00 00 00 00 00 00 00 00 0
0 00 00 00 00 00 00 01 58

43 msg bytes + padding = 64 bytes = 1 block of 512 bits

2 · WORKING REGISTERS (FINAL BLOCK)

h0	h1	h2	h3	h4
2fd4e1c6	7a2d28fc	ed849ee1	bb76e739	1b93eb12

3 · ROUND TRACE (FINAL BLOCK · A B C D E)

Figure 1: SHA-1 digest demonstrator

Algorithm

SHA-1 is a one-way digest, so there is no decrypt. The two core methods are padding the message and compressing each block into the running hash.

```

1 Function Pad(message):
2   bits ← length of message in bits
3   append the byte 0x80 to message
4   while length mod 512 ≠ 448 bits do
5     | append a 0x00 byte
6   append bits as a 64-bit big-endian integer           // now a multiple of 512
7   return message split into 512-bit blocks

```

```

1 Function Digest(message):
2   (h0..h4) ← (67452301, EFCDA89, 98BADCFE, 10325476, C3D2E1F0)
3   foreach block in Pad(message) do
4     w[0..15] ← the sixteen 32-bit words of block
5     for t ← 16 to 79 do
6       | w[t] ← (w[t-3] XOR w[t-8] XOR w[t-14] XOR w[t-16]) rot_left 1
7     a, b, c, d, e ← (h0, h1, h2, h3, h4)
8     for t ← 0 to 79 do
9       | (f, k) ← round constants for t           // 4 stages of 20 rounds
10      | tmp ← (a rotl 5) + f + e + k + w[t]         // all mod 232
11      | (e, d, c, b, a) ← (d, c, b rotl 30, a, tmp)
12      | h0..h4 ← (h0+a, h1+b, h2+c, h3+d, h4+e)   // mod 232
13   return h0 || h1 || h2 || h3 || h4           // concatenate: 160-bit digest

```

The per-round (f, k) come in four stages of twenty rounds:

Rounds t	f(b, c, d)	k
0 - 19	(b AND c) OR (NOT b AND d)	5A827999
20 - 39	b XOR c XOR d	6ED9EBA1
40 - 59	(b AND c) OR (b AND d) OR (c AND d)	8F1BBCDC
60 - 79	b XOR c XOR d	CA62C1D6

Output

Digest — message abc (3 bytes)

Padding: append 0x80, fill with zeros, then the 64-bit length 0x18 = 24 bits. The message is short, so it forms a single 512-bit block:

```

61 62 63 80 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 18

```

The block is loaded as sixteen words (w0 = 61626380, w1..w14 = 0, w15 = 00000018), expanded to 80 words, then stirred through the 80 rounds. The five registers start at the fixed constants and evolve each round:

Round t	a	b	c	d	e
init	67452301	EFCDAB89	98BADCFE	10325476	C3D2E1F0
0	0116FC33	67452301	7BF36AE2	98BADCFE	10325476
1	8990536D	0116FC33	59D148C0	7BF36AE2	98BADCFE
2	A1390F08	8990536D	C045BF0C	59D148C0	7BF36AE2
3	CDD8E11B	A1390F08	626414DB	C045BF0C	59D148C0
...	...	...	...	...	...

After 80 rounds, the registers are added back into the running hash, giving the final five words:

Word	Value
h0	A9993E36
h1	4706816A
h2	BA3E2571
h3	7850C26C
h4	9CD0D89D

Concatenating h0..h4 gives the 160-bit digest.

Result: `abc` -> `a9993e364706816aba3e25717850c26c9cd0d89d`

Avalanche — messages `...lazy dog` vs `...lazy cog`

A single-letter change scrambles most of the digest, showing the one-way mixing:

Message	Digest
The quick brown fox jumps over the lazy dog	<code>2fd4e1c67a2d28fced849ee1bb76e7391b93eb12</code>
The quick brown fox jumps over the lazy cog	<code>de9f2c7fd25e1b3afad3e85a0bd17d9b100db4b3</code>

One changed character flips 87 of the 160 output bits (about 54%). There is no inverse operation: the digest cannot be turned back into the message.